

期望方差与抽样误差实验

目标和完成证据

本实验要识别“抽了多少个独立单位”，并核对它如何进入平均损失的误差。提交手算、重跑结果、错误SE诊断与失效说明。合成例子不用于评价真实模型。先修017离散部分；连续选做另需010。

固定图和Notebook仅接受原始默认配置与对应outputs，先检查再运行；自定义模拟请用脚本--config及--output另存，图文不自动适配。

只用CPU和Python标准库即可运行核心脚本与测试。Notebook需要Jupyter内核；观看内嵌图片需要IPython，图片文件已提供，无需重新生成。可选重建图需要NumPy和matplotlib，正文中的统计运算不依赖它们。

一 先手算再运行

四个等权编号的值为0、0、2、6。用一张纸写下 $E[X]$ 、 $E[X^2]$ 、 $\text{Var}(X)$ ，以及 $n=64$ 个IID样本的真实均值SE。然后换成8个独立组、每组复制8次，重新计算SE。不要看到“64行”就套 $\sigma/\sqrt{64}$ 。

答案关键值依次为2、10、6、 $\sqrt{(3/32)} \approx 0.30619$ 和 $\sqrt{(3/4)} \approx 0.86603$ 。先写自己的推导及根号内的量。单行分布相同，变的是整批联合结构。

二 检查环境与输入

打开本单元目录，可使用既有Python环境，或按下面建议建立独立Anaconda环境：

```
conda env create -f environment.yml
conda activate dl-unit-018
python --version
python experiment.py --output outputs
python test_experiment.py
jupyter lab
```

已实测Python3.12.14新进程脚本、测试及真实InProcessKernel顺序执行。上面的全新Anaconda安装、浏览器界面、socket内核与跨平台流程未实测。

读data/config.json: seed=20261004, repeats=4000, sample_sizes为16、32、64、128, copies=8。atoms列表有两个0，代表不同原子编号。随机发生器是局部Random实例。data/group_observations.csv是另一份固定手算样本，不是输出中挑出的最好一次。

运行脚本会写四个文件：results.json存精确手算及配置，sampling_trials.csv存全部32000批结果，sampling_summary.csv存8行摘要，environment.json存环境。重跑同一输出目录会覆盖这些文件；保留旧实验请指定不同目录。输入配置与数据不改动。Notebook改写notebook_outputs以便比较，仓库仅随附一份脚本输出。

三 精确矩与非线性变换

在Python或Notebook执行：

```
from fractions import Fraction
from experiment import finite_moments, covariance, sample_statistics
mu, second, variance = finite_moments([0, 2, 6], [2, 1, 1])
assert (mu, second, variance) == (Fraction(2), Fraction(10), Fraction(6))
assert second != mu * mu
```

counts参数是非负整数质量，总和必须正；函数自己归一化。数值输入支持整数、Fraction或分数字符串，拒绝布尔值、二进制float及非有限字符串。把counts改为[1,1,1]后，均值与方差会改变，因为你改了模型，不能把这称作数值误差。

接着计算X在-1、0、1均匀分布且 $Y=X^2$ 的协方差：

```
assert covariance([-1, 0, 1], [1, 0, 1], [1, 1, 1]) == 0
```

再写下 $P(Y=0|X=0)=1$ 与 $P(Y=0)=1/3$ 。协方差0不能证明独立。

四 把同样的8组复制成64行

先读取固定组数据并保留ID。数组或列表的位置不是抽样设计的替代品。

```
import csv
from pathlib import Path
with Path('data/group_observations.csv').open() as stream:
    rows = list(csv.DictReader(stream))
groups = [int(row['loss']) for row in rows]
mean, s2, se2 = sample_statistics(groups)
assert (mean, s2, se2) == (Fraction(2), Fraction(48, 7), Fraction(6, 7))
```

现在复制每个组，并计算错误逐行估计：

```
copied_rows = [x for x in groups for _ in range(8)]
row_mean, row_s2, row_se2 = sample_statistics(copied_rows)
assert row_mean == mean
assert row_s2 == Fraction(128, 21)
assert row_se2 == Fraction(2, 21)
assert se2 / row_se2 == 9
```

这里估计方差之比为9，不是8。8是两种机制真实总体均值方差之比；9来自本例把同一组样本先后用7与63作样本方差分母。两者比较对象不同。正确处理是以原8组计算SE，得到 $\sqrt{(6/7)}$ ，而不是把错误值乘一个没说明来源的修正因子。

五 精确枚举和4000批模拟

精确枚举用整数计数卷积累加和分布，不使用随机数：

```
from experiment import exact_mean_distribution
law = exact_mean_distribution([0, 0, 2, 6], 2)
assert sum(p for x, p in law) == 1
assert sum(x * p for x, p in law) == 2
assert sum((x - 2)**2 * p for x, p in law) == 3
```

n=1、4、16、64的图来自精确质量。教学算法仅支持有界整数原子及 $n \leq 64$ 。

再打开sampling_summary.csv。n=64的IID与复制机制，4000批均值的样本SD应分别约0.30820与0.87756；理论值分别约0.30619与0.86603。模拟平均约2.00946与2.01769，对应MCSE约0.00487与0.01388，仅表示外层模拟平均的不稳定程度。

三种数字不要混读：empirical_sd是R批均值的样本SD；rms_naive_se是每批错误逐行SE平方后平均再开方；rms_unit_correct_se按真实独立单位计算。复制机制n=64时，后两项约0.28948与0.86843。单批估计本身也随机，不应强求每次等于理论值。

六 故意让估计失效

1. 只保留8个组不变，把每组复制数从8增至32。真实均值与组SE不变，但天真的逐行SE继续缩小。用手算证明，不必把模拟配置的n和组数同时改动后混称“只复制”。
2. 令所有记录永久等于一次抽到的Z。判断均值在n增加时是否接近2，并给概率反例。
3. 对Bernoulli($p=0.001$)， $n=100$ ，计算全零概率 0.999^{100} 。全零批次给样本SE=0，写出为什么不能报告“概率已确定为0”。
4. 将配置copies改为0、将某个n改成不能整除copies、或将repeats改成布尔值。脚本应拒绝并保留旧输出。测试文件会自动在临时目录检查这些路径，不要直接修改唯一保留的实验记录。

七 重跑与解释报告

重启Notebook内核后按顺序运行全部单元，检查执行计数连续、没有错误输出，再比较notebook_outputs与outputs四个文件。脚本从其他工作目录运行时应仍能找到自身的配置；Notebook示例的相对CSV路径约定是内核在本单元目录。若另处打开，请先切回本单元目录。

报告说明目标分布、独立单位、所算误差对象与假设。不把模拟包装成算法优劣，也不把固定测试集换seed的波动当作新总体抽样误差。